

Disk Scheduling Algorithms

Disk scheduling algorithms manage how data is read from and written to a computer's hard disk. These algorithms help determine the order in which disk read and write requests are processed.

- Disk scheduling is also known as I/O Scheduling.
- The main goals of disk scheduling are to optimize the performance of disk operations, reduce the time it takes to access data and improve overall system efficiency.
- Common disk scheduling methods include First-Come, First-Served (FCFS), Shortest Seek Time First (SSTF), SCAN, C-SCAN, LOOK, and C-LOOK.

Importance of Disk Scheduling in Operating System

- Multiple I/O requests may arrive by different processes and only one I/O request can be served at a time by the disk controller. Thus other I/O requests need to wait in the waiting queue and need to be scheduled.
- Two or more requests may be far from each other so this can result in greater disk arm movement.
- Hard drives are one of the slowest parts of the computer system and thus need to be accessed in an efficient manner.

Key Terms Associated with Disk Scheduling

- **Seek Time:** Time taken to move the disk arm to the track where data is located.
- **Rotational Latency:** Time taken for the desired sector to rotate under the read/write head.
- **Transfer Time:** Time taken to actually read or write the data, depending on disk speed and data size.

$$\text{Disk Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$

$$\text{Total Seek Time} = \text{Total head Movement} * \text{Seek Time}$$

Disk Access Time and Disk Response Time

Disk Response Time

- **Response Time:** The time a request waits before its I/O operation starts.
- **Average Response Time:** The mean waiting time of all requests.
- **Variance in Response Time:** How much individual waiting times differ from the average.

Goals of Disk Scheduling Algorithms

- Minimize Seek Time
- Maximize Throughput
- Minimize Latency
- Ensuring Fairness
- Efficiency in Resource Utilization

Disk Scheduling Algorithms

There are several Disk Scheduling Algorithms. We will discuss in detail each one of them.

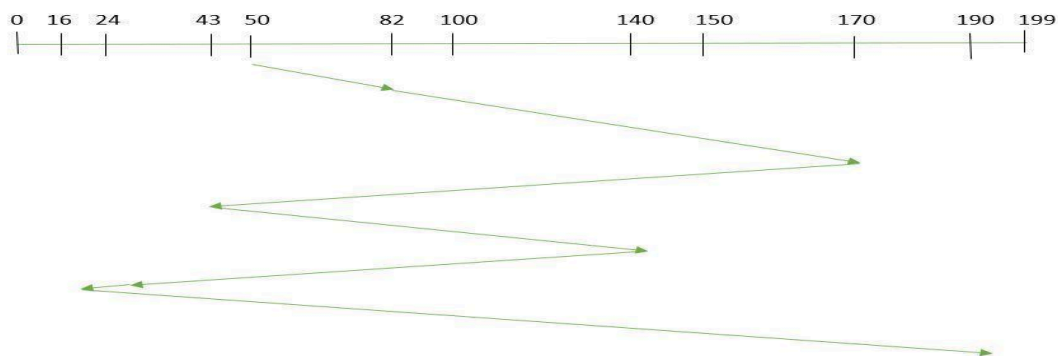
- FCFS (First Come First Serve)
- SSTF (Shortest Seek Time First)
- SCAN
- C-SCAN
- LOOK
- C-LOOK

1. FCFS (First Come First Serve)

FCFS is the simplest of all Disk Scheduling Algorithms. In FCFS, the requests are addressed in the order they arrive in the disk queue. Let us understand this with the help of an example.

First Come First Serve

Suppose the order of request is- (82,170,43,140,24,16,190) and current position of Read/Write head is: 50



$$\begin{aligned} &\text{So, total overhead movement (total distance covered by the disk arm)} \\ &= \\ &(82-50) + (170-82) + (170-43) + (140-43) + (140-24) + (24-16) + (190-16) \\ &= 642 \end{aligned}$$

Advantages of FCFS

Here are some of the advantages of First Come First Serve.

- Every request gets a fair chance
- No indefinite postponement

Disadvantages of FCFS

Here are some of the disadvantages of First Come First Serve.

- Does not try to optimize seek time

- May not provide the best possible service

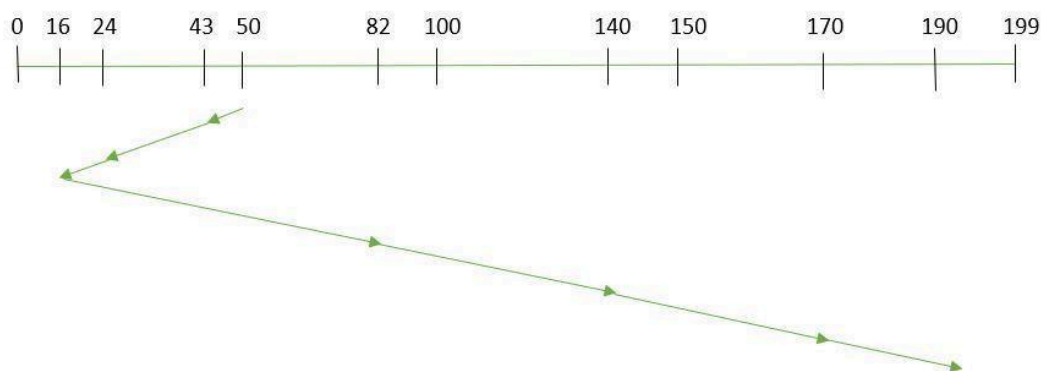
2. SSTF (Shortest Seek Time First)

In SSTF (Shortest Seek Time First), requests having the shortest seek time are executed first. So, the seek time of every request is calculated in advance in the queue and then they are scheduled according to their calculated seek time. As a result, the request near the disk arm will get executed first. SSTF is certainly an improvement over FCFS as it decreases the average response time and increases the throughput of the system. Let us understand this with the help of an example.

Example:

Shortest Seek Time First

Suppose the order of request is- (82,170,43,140,24,16,190) and current position of Read/Write head is: 50



$$\begin{aligned} \text{total overhead movement (total distance covered by the disk arm)} &= \\ &= (50-43) + (43-24) + (24-16) + (82-16) + (140-82) + (170-140) + (190-170) \\ &= 208 \end{aligned}$$

Advantages of Shortest Seek Time First

Here are some of the advantages of Shortest Seek Time First.

- The average Response Time decreases
- Throughput increases

Disadvantages of Shortest Seek Time First

Here are some of the disadvantages of Shortest Seek Time First.

- Overhead to calculate seek time in advance

- Can cause Starvation for a request if it has a higher seek time as compared to incoming requests
- The high variance of response time as SSTF favors only some requests

3. SCAN

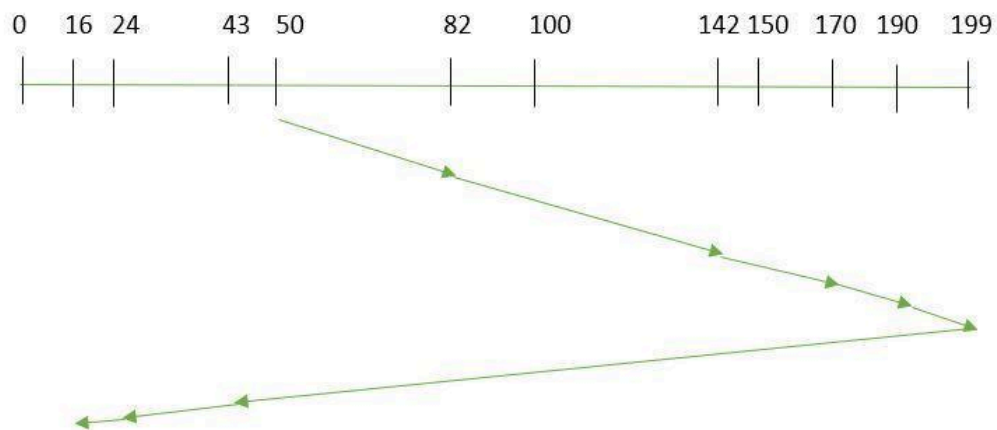
In the SCAN algorithm the disk arm moves in a particular direction and services the requests coming in its path and after reaching the end of the disk, it reverses its direction and again services the request arriving in its path. So, this algorithm works as an elevator and is hence also known as an **elevator algorithm**. As a result, the requests at the midrange are serviced more and those arriving behind the disk arm will have to wait.

Example:

SCAN Algorithm

Suppose the requests to be addressed are-82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "**towards the larger value**".

Therefore, the total overhead movement (total distance covered by the disk arm) is calculated as



$$= (199-50) + (199-16) = 332$$

Advantages of SCAN Algorithm

Here are some of the advantages of the SCAN Algorithm.

- High throughput
- Low variance of response time
- Average response time

Disadvantages of SCAN Algorithm:

- Head may move to disk end unnecessarily
- High waiting time for some requests
- New requests may wait longer

4. C-SCAN

In the SCAN algorithm, the disk arm again scans the path that has been scanned, after reversing its direction. So, it may be possible that too many requests are waiting at the other end or there may be zero or few requests pending at the scanned area.

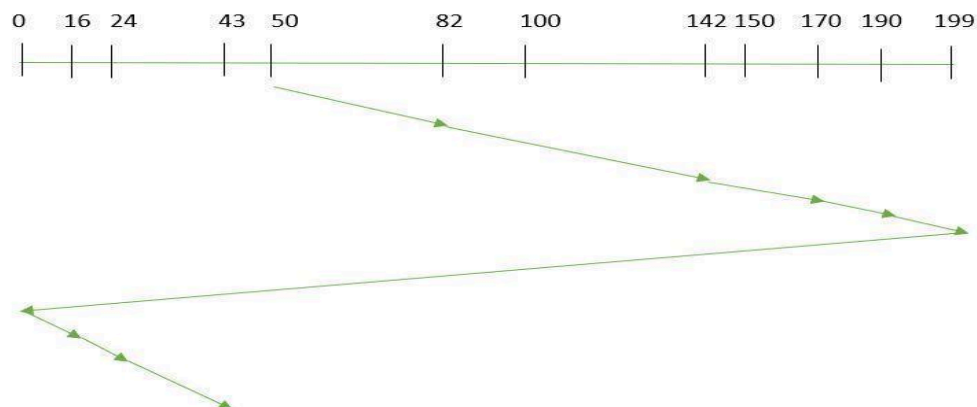
These situations are avoided in the C-SCAN algorithm in which the disk arm instead of reversing its direction goes to the other end of the disk and starts servicing the requests from there. So, the disk arm moves in a circular fashion and this algorithm is also similar to the SCAN algorithm hence it is known as C-SCAN (Circular SCAN).

Example:

Circular SCAN

Suppose the requests to be addressed are- 82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "**towards the larger value**".

So, the total overhead movement (total distance covered by the disk arm) is calculated as:



$$=(199-50) + (199-0) + (43-0) = 391$$

Advantages of C-SCAN Algorithm:

- Eliminates starvation of requests
- Better performance for heavy disk load
- More fair than SCAN algorithm

Learn more in detail: [C-SCAN](#)

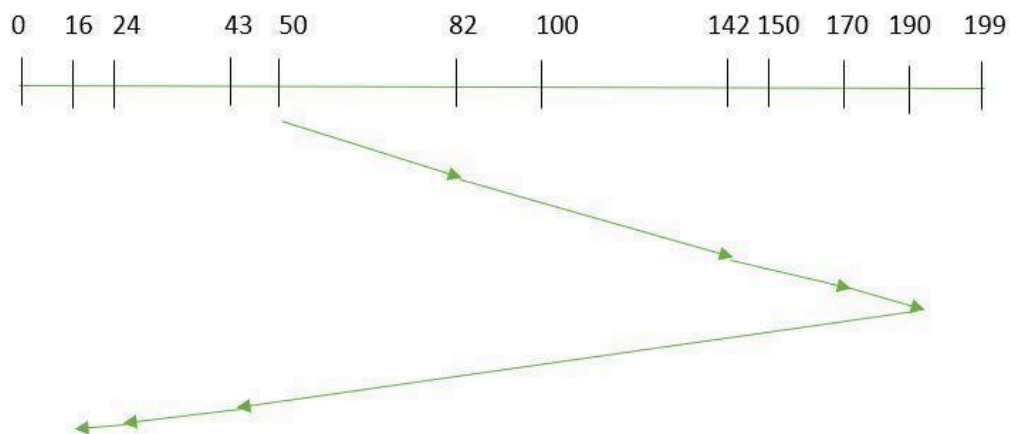
Disadvantages of C-SCAN Algorithm:

- Extra head movement due to return to start
- Increased seek time compared to SCAN in light load
- More overhead because of circular movement

5. LOOK

LOOK Algorithm is similar to the SCAN disk scheduling algorithm except for the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Example:



LOOK Algorithm

Suppose the requests to be addressed are- 82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value".

So, the total overhead movement (total distance covered by the disk arm) is calculated as:

$$= (190-50) + (190-16) = 314$$

Advantages

- **Reduced Unnecessary Movement:** The disk arm only goes as far as the last request in each direction, avoiding travel to the disk's physical end (unlike SCAN).
- **Faster Response:** Less head movement leads to quicker service for requests.

Disadvantages of LOOK Algorithm:

- Waiting time can still be high for some requests
- Performance depends on request distribution

- Not completely fair to newly arrived requests

Learn more in detail: [LOOK](#)

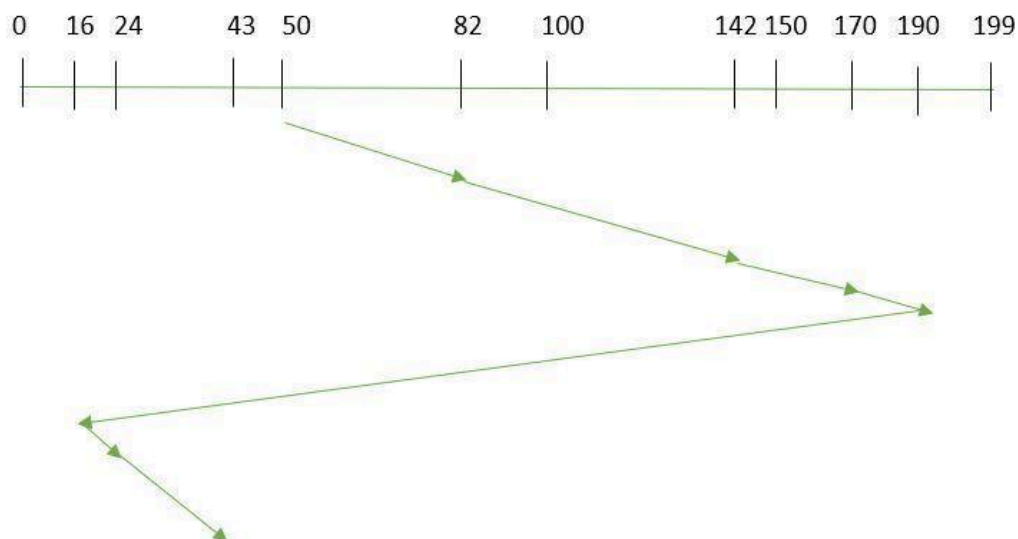
6. C-LOOK

As LOOK is similar to the SCAN algorithm, in a similar way, C-LOOK is similar to the CSCAN disk scheduling algorithm. In CLOOK, the disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request. Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Example: Suppose the requests to be addressed are-82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value"

C-LOOK

So, the total overhead movement (total distance covered by the disk arm) is calculated as



$$= (190-50) + (190-16) + (43-16) = 341$$

Advantages:

- **Uniform Wait Time:** Requests are serviced in a circular manner, so waiting times are more predictable and fair.
- **Reduced Head Movement:** The arm only goes as far as the last request in one direction, then jumps back, saving time compared to C-SCAN.

Disadvantages of C-LOOK Algorithm:

- Extra head movement due to circular scanning

- Increased seek time compared to LOOK under light load
- Newly arrived requests may have to wait for a full cycle